The situation

SAP Business Data Cloud delivers data products into Snowflake as catalog-linked databases. Consumers on the reference accounts measured data arriving hours to a day behind business events because their Gold layers re-read the entire share on a schedule - nothing signalled what had changed.

Snowflake's standard change-data-capture mechanisms - streams, CHANGES(), and change tracking - are refused on these objects. They are Iceberg tables on an external SAP-managed catalog, and that single fact explains why a different approach is needed.

What was found

Two change signals exist. Which one applies depends on the kind of data product:

Track	Signal	Deletes
Track 1 Custom products	Source-system timestamp (e.g. ZZLASTCHGDATE), configured by the pipeline owner. Found on 59 objects across the reference account.	No operation column. Invisible on this track. Reconcile row counts on a schedule.
Track 2 Standard products	Four SAP-generated columns: __OPERATION_TYPE, __TIMESTAMP, and two hash-suffixed columns. Nothing to configure in the source system.	Operation column present. Delete code not observed and not confirmed by SAP documentation (unverified). Reconcile row counts on a schedule.

On Track 1, a watermark predicate returned changed rows while scanning 0.00 MB of a 7,035.80 MB object containing 108 million rows. Polling the watermark cost 0.00 MB and 47-79 ms. On Track 2, polling the current timestamp as raw text cost 0.000 MB in 2 ms, answered from Iceberg manifest statistics. Dynamic tables with REFRESH_MODE = INCREMENTAL were proven end-to-end on both tracks; a no-change refresh returned NO_DATA in under one second.

The decision

1. Prefer standard data products where they exist. Only they carry an operation column, so only they can represent a delete.

2. On both tracks, use dynamic tables with REFRESH_MODE pinned explicitly to INCREMENTAL. The watermark is a trigger and exception path, not the engine. Never leave REFRESH_MODE as AUTO: it falls back silently to full-rebuild mode with no warning.

3. Point capture at the catalog-linked database. Never at a copy. Validate the standard metadata before building on it.

4. Reconcile row counts on a schedule on both tracks. Deletes are unrepresented on Track 1 and unverified on Track 2.

The risk

Every local copy of a standard data product on the reference account had corrupted metadata. On one copy, the operation column held media titles. On two others, it held customer identifiers across all rows (735,305 of 735,305 and 557,693 of 558,011 respectively). None of this failed loudly. It produced a wrong incremental result quietly and the defect propagated into downstream medallion layers.

This is not theoretical: three separate copies on a live account were all corrupted. The safeguard is to point capture at the catalog-linked database itself and validate the metadata columns. The readiness probe does both.

What to do now

Step	Action
1	Run probes/cdc_readiness.py against your tenant. It classifies every object by track and validates the metadata in under a minute. Review the verdict before deploying anything.
2	Create Gold-layer dynamic tables with REFRESH_MODE = INCREMENTAL stated explicitly. Never AUTO. One wrong setting cascades downstream.
3	Deploy sql/03_monitoring.sql - specifically VW_RELOAD_DETECTION. The pattern rests on the connector merging; a switch to reload raises no error and is only visible through this monitor.
4	Reconcile row counts on a regular schedule. Dynamic tables capture physical deletes; the watermark path does not. Either way, an independent count check is the safety net.